

Track Malfunctions in Flatland

Baruch Shteken Lena Friedek

Railway Scheduling, Universität Potsdam

21st Sep 2026

Malfunctions in Flatland

- solve.py handles train malfunctions.
- Base encoding for scheduling and forwarding information; secondary encoding for rescheduling (Gilka and Murphy).
- Information forwarded:

- actions as constraints;

```
:- not action(train(0),move_forward,22).  
:- not action(train(1),move_forward,22).  
:- not action(train(1),wait,24).
```

- the malfunction atom;

```
malfunction(1,6,23).
```

- planned_action.

```
planned_action(train(0),move_forward,0).  
planned_action(train(0),move_forward,1).
```

Track Malfunctions

- real-world issue
- negatively affects punctuality
- reasons: weather, fires, trespassing, repairs, high-frequent use

Our Contribution

- create malfunction (randomly/deterministically)
- forwarding additional information
- malfunction logging
- visualization in Flatland

Create a Track Malfunction

```
1 def create_trkmalf(self, timestep, predefined_mals):
2
3     if not predefined_mals:
4         malfunction_ind = random.random()
5
6         if malfunction_ind > 0.99:
7             malfunction_cell = tuple(random.choice(self.grid))
8             malfunction_duration = random.randint(2,20)
9             mal_timestep = timestep
10
11         else:
12             return ([])
13
14     ...
15     self.track_malfunctions.append((malfunction_cell,
16                                     malfunction_duration))
17     self.track_record.append((malfunction_cell, mal_timestep,
18                               malfunction_duration))
19     return([(malfunction_cell, malfunction_duration)])
```

Additional Forwarded Information

- track malfunction

```
track_malfunction((17, 19), 21, 13).
```

- position constraint rules

```
:- position( _, (17, 19), 14, _).
```

```
:- position( _, (17, 19), 15, _).
```

- planned position

```
planned_position(0, (20, 3), 15, s).
```

```
planned_position(1, (28, 25), 15, n).
```

- current position

```
current_position(0, (19, 3), 14, s).
```

```
current_position(1, (29, 25), 14, n).
```

Secondary Encoding

- add figures for base and secondary
-

Rescheduling

The basis of our secondary encoding is to determine which trains are affected by the malfunction and reschedule only them unless there is a collision and then reschedule also the trains affected by the collision

- Check if there is a consequence

```
consequence(train(ID), P, T) :- planned_position(ID, P, T, _),  
track_malfunction(P, Dur_mal, T_mal), Dur_mal + T_mal >= T.
```

- Use as many planned trains as possible

```
{planned(ID)}:- train(ID), not consequence(train(ID), _, _).  
#maximize { 1@3, ID : planned(ID)}.
```

- Recalculate schedule for unplanned trains

```
calculate(ID) :- consequence(train(ID), _, _).  
calculate(ID) :- train(ID), not planned(ID).
```

- Using heuristic speeds up the process

```
#heuristic planned(ID): train(ID). [100, true]
```


Secondary Encoding vs Secondary Encoding with Time Prioritization

The encodings are different only in their optimization rule prioritization

- Secondary Encoding

```
#maximize { 1@3, ID : planned(ID)}.
```

```
#minimize { 1@2, ID, T : action(train(ID), _, T) }.
```

- Secondary Encoding with Time Prioritization

```
#maximize { 1@2, ID : planned(ID)}.
```

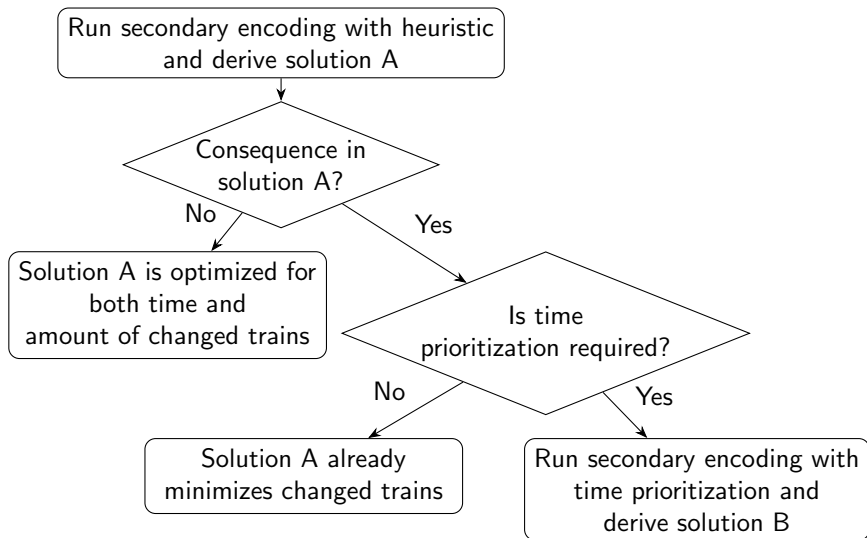
```
#minimize { 1@3, ID, T : action(train(ID), _, T) }.
```

Experiments

- Effect of the Heuristic
- Base Encoding vs. Secondary Encoding
- Base Encoding vs. Secondary Encoding with Time Prioritization
- Heuristic vs. Time Prioritization

- Time optimization and base encoding outputs different schedule than secondary encoding
- Secondary encoding with heuristic is better than no heuristic
- Secondary encoding with heuristic is the fastest
- Time prioritization is faster than base encoding when there is a consequence

Discussion



Future Work

- add figures for base and secondary
-